Module: Computer and Programming Foundations

Next module: Python Setup and First Programs

Learning Outcomes

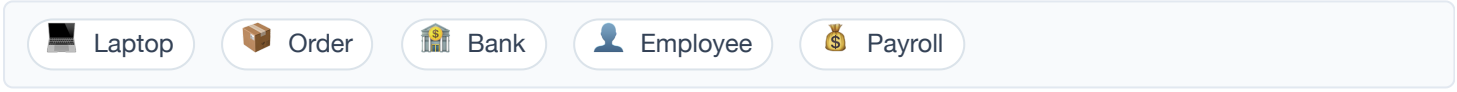
By the end of this module, learners will be able to:

- Explain what a computer, operating system, and file system are in simple English
- Navigate folders and files using VS Code and the terminal (Windows/macOS/Linux)
- Describe what programming is and how it differs from an algorithm
- Draw a simple client–server diagram for a web application
- Name common developer roles in the Indian IT industry and what each role does
- Set up VS Code with Python extensions and run basic terminal commands
- Create a personal learning roadmap note for their full-stack journey

---

What is a computer?

A **computer** is a machine that takes **input**, processes it using rules, and gives **output**. When you open a student portal on your phone, you type a roll number (input), the server checks the database (processing), and shows your marks (output).



Where used?
- College apps — student portal login, marks screens, lab machines
- **TechMart** / CampusCart — checkout flow (browser client → server → DB)
- **SecureBank** / UPI — ATM withdrawal; app request → bank server
- **AcmeHR** — employee tools that run on company laptops
- **MedCare** — appointment desks using folders, terminals, and web portals

Think of it like a kitchen:

```
[You give ingredients] → [Chef follows recipe] → [Dish served]
     INPUT                  PROCESSING              OUTPUT
```

What is an operating system (OS)?

The **operating system** is the manager of your laptop. Windows, macOS, and Linux are operating systems. The OS:

- Starts your laptop when you press the power button
- Shows the desktop and file explorer
- Runs programs (browser, VS Code, Python)
- Manages files and folders on your hard disk

Without an OS, your laptop is just hardware with no way for you to interact easily.

What are files and folders?

A **folder** (also called a **directory**) is like a drawer in a cupboard. A **file** is a document inside that drawer.

Example structure for this course:

```
FULL_STACK_PYTHON/
├── 01_Foundations/
├── 02_Core_Python/
└── projects/
    └── hello.py
```

A **file path** is the full address of a file, for example:

- Windows: `C:\Users\Ravi\projects\hello.py`
- macOS/Linux: `/Users/ravi/projects/hello.py`

File extension (the part after the dot) tells the OS what kind of file it is:

Extension	Meaning
`.py`	Python program
`.txt`	Plain text
`.md`	Markdown notes
`.html`	Web page
`.json`	Data in JSON format

What is programming?

Programming is writing instructions in a language that a computer can follow. These instructions are saved in **source code** files.

An **algorithm** is a step-by-step plan to solve a problem — you can write it in plain English or on paper. **Code** is the algorithm written in a programming language like Python.

```
Algorithm (English):          Code (Python — coming in next module):
1. Ask user name              name = input("Enter name: ")
2. Greet the user             print("Hello,", name)
```

You do not need to memorize code on day one. First understand the *problem* and the *steps*.

Client–server and the browser

Most apps you use daily follow a **client–server** model:

```
[Client]  ---- HTTP Request ---->  [Server]
  (Browser / Mobile App)            (Python/Django app on cloud)
[Client]  <--- HTTP Response ----  [Server]
              (HTML, JSON, data)
```

- **Client:** Your Chrome browser or phone app — it asks for data
- **Server:** A computer in a data centre (AWS, Azure, GCP) — it processes requests and sends replies

- **Request:** "Show me student ID 101's profile"
- **Response:** JSON or HTML page with the profile

When you open `https://example.com/login`, your browser (client) sends a request; the server checks credentials and sends back the dashboard page.

> **Interview:** "What happens when you type a URL?" → DNS finds the server IP → browser sends HTTP request → server returns HTML/JSON → browser renders.

Developer roles in India (entry-level view)

Role	What they build	Typical tools	Example in training
Frontend Developer	Screens users see and click	HTML, CSS, JavaScript, React	Student portal UI
Backend Developer	Logic, APIs, database	Python, Django, MySQL	Employee login API
Full Stack Developer	Both UI and server	Python + React + MySQL	Expense tracker end-to-end
QA / Tester	Finds bugs before release	Test cases, Selenium	Test login form
DevOps Engineer	Deploys and monitors apps	Git, Docker, AWS	Put app on server
Data Analyst / Python Dev	Reports and scripts	Python, pandas, SQL	Sales report script

In a 16-week full-stack program, you touch frontend, backend, database, and deployment — that is why the role is called **full stack**.

VS Code and terminal basics

VS Code (Visual Studio Code) is a free **IDE** (Integrated Development Environment) — an editor with tools for coding, debugging, and running programs.

The **terminal** (also called command line or shell) is a text-based way to talk to your OS. Developers use it to run programs, create folders, and use Git.

Important Concepts and Definitions

Term	Definition
Hardware	Physical parts: CPU, RAM, keyboard, screen
Software	Programs and apps that run on hardware
OS (Operating System)	System software that manages hardware and files
File	A named collection of data stored on disk
Directory / Folder	A container that holds files and other folders
Path	Location of a file or folder in the file system
Algorithm	Ordered steps to solve a problem

Term	Definition
Source code	Human-readable program text
Compiler / Interpreter	Translates code to machine instructions (Python uses an interpreter)
Client	Device or app that requests services
Server	Machine that provides services and data
HTTP	Protocol (rules) for web request/response
IDE	Editor with coding tools built in
Terminal / CLI	Command-line interface to the OS
Repository (repo)	Project folder tracked by Git (introduced later)

Step-by-Step Notes (With Examples)

Step 1: Know your machine

Check your laptop specs (Settings → About):

- **CPU:** Brain of the computer (Intel i5, Apple M1, etc.)
- **RAM:** Short-term memory while programs run (8 GB minimum recommended)
- **Storage:** Long-term space for files (SSD preferred)

> **Note:**
> - Wrong: Skipping OS / hardware basics and jumping straight to Python.
> - Prefer: One day on CPU, RAM, storage, and files — it saves weeks of confusion later.

Try these (2 programs):

Program 1 — Specs checklist note

```
# My Laptop Specs — Priya (Hyderabad)

- OS: macOS / Windows / Linux
- CPU: _________________
- RAM: _____ GB (need ≥ 8 GB for VS Code + browser)
- Storage free: _____ GB
- Ready for course? YES / NO
```

Program 2 — Hardware vs software table

```
| Item              | Hardware or Software? |
|-------------------|-----------------------|
| Keyboard          | Hardware              |
| Chrome browser    | Software              |
| SSD drive         | Hardware              |
| VS Code           | Software              |
| Python 3          | Software              |
```

Step 2: Create your course folder structure

Open terminal and run (macOS/Linux):

```
# Create main project folder
mkdir -p ~/ksr_python_fullstack/projects
cd ~/ksr_python_fullstack
ls
```

Windows (Command Prompt or PowerShell):

```
mkdir C:\Users\YourName\ksr_python_fullstack\projects
cd C:\Users\YourName\ksr_python_fullstack
dir
```

Note:
- Wrong: Saving course files randomly on the Desktop; using spaces in folder names (`my project`).
- Prefer: One root folder (`ksr_python_fullstack`) and `snake_case` / `kebab-case` names (`my_project`).

Try these (2 programs):

Program 1 — Full course tree (macOS/Linux)

```
mkdir -p ~/ksr_python_fullstack/{notes,projects,labs}
cd ~/ksr_python_fullstack
pwd
ls
```

Expected Output: (paths vary by user)

```
/Users/you/ksr_python_fullstack
labs
notes
projects
```

Program 2 — Nested lab folder

```
cd ~/ksr_python_fullstack
mkdir -p labs/day01_workspace
cd labs/day01_workspace
pwd
ls
```

Step 3: Install and open VS Code

1. Download from https://code.visualstudio.com
2. Install **Python** extension (by Microsoft)
3. **File** → **Open Folder** → select `ksr_python_fullstack`
4. Create a file: `notes/day01_foundations.txt`

Note:
- Wrong: Opening a single file in VS Code, then wondering why paths and terminals feel broken.
- Prefer: Always **File → Open Folder** on the project root.

Try these (2 programs):

Program 1 — First notes file

```
# Day 01 Foundations — Ravi

What I learned:
1. Computer = input → process → output
2. OS manages hardware and files
3. Terminal is how developers talk to the OS
```

Program 2 — Glossary starter

```
# Glossary

- **Path:** full address of a file (e.g. `/Users/ravi/projects/hello.py`)
- **Extension:** part after the dot (`.py`, `.md`, `.html`)
- **IDE:** editor with coding tools (VS Code)
```

Step 4: Essential terminal commands

Command	macOS/Linux	Windows	Purpose
List files	`ls`	`dir`	See what is in current folder
Change directory	`cd folder_name`	`cd folder_name`	Move into a folder
Go up one level	`cd ..`	`cd ..`	Move to parent folder
Print working directory	`pwd`	`cd` (shows path)	Show current location
Create folder	`mkdir name`	`mkdir name`	New directory
Clear screen	`clear`	`cls`	Clean terminal view
Check Python (later)	`python3 --version`	`python --version`	Verify Python install

Annotated demo — navigate like a developer:

```
# You are here: /Users/priya
cd ksr_python_fullstack          # enter course folder
mkdir 01_Foundations             # create module folder
cd 01_Foundations                # go inside it
pwd                              # confirm path → .../01_Foundations
ls                               # list contents (may be empty)
```

Expected Output:

```
/Users/priya/ksr_python_fullstack/01_Foundations
```

(`ls` may print nothing if the folder is empty.)

Remember: `pwd` = “where am I?” — run it after every `cd` until paths feel natural.

Note:
- Wrong: Avoiding the terminal because it looks like “hacking” movies.
- Prefer: Run only the commands shown in class — `ls` / `cd` are safe navigation tools.

Try these (2 programs):

Program 1 — Navigate and confirm path

```
cd ~
pwd
cd ksr_python_fullstack
mkdir -p notes
cd notes
pwd
ls
```

Program 2 — Up one level drill

```
cd ~/ksr_python_fullstack/notes
pwd
cd ..
pwd
ls
```

Common Error: Typing `cd ksr_python_fullstack` from the wrong folder → `No such file or directory`. Use `cd ~` first, or the full path `~/ksr_python_fullstack`.

Step 5: Algorithm vs code — student login example

Scenario: A college wants a simple login check.

Algorithm (pseudocode on paper):

```
START
  DISPLAY "Enter roll number"
  READ roll_number
  DISPLAY "Enter password"
  READ password
  IF roll_number is valid AND password matches
    DISPLAY "Welcome to student portal"
  ELSE
    DISPLAY "Invalid login"
  END IF
END
```

Flowchart (text form):

```
        [Start]
           |
    [Ask roll number]
           |
    [Ask password]
           |
    [Valid?]---No--->[Show error]--->[End]
```

```
       |
      Yes
       |
 [Show portal home]
       |
      [End]
```

You will write the Python version in Module 03 (Flow Control). Today, focus on *logic*, not syntax.

> **Note:**
> - Wrong: Mixing up *algorithm* (English steps) with *code* (Python syntax).
> - Prefer: Write steps in plain English / pseudocode first; code comes later.

Try these (2 programs):

Program 1 — SecureBank ATM withdraw algorithm

```
START
  DISPLAY "Enter PIN"
  READ pin
  IF pin is correct
    DISPLAY "Enter amount (₹)"
    READ amount
    IF amount <= balance
      DISPLAY "Dispense cash"
      SET balance = balance - amount
    ELSE
      DISPLAY "Insufficient balance"
    END IF
  ELSE
    DISPLAY "Wrong PIN"
  END IF
END
```

Program 2 — Exam pass/fail algorithm

```
START
  READ marks
  IF marks >= 35
    DISPLAY "Pass"
  ELSE
    DISPLAY "Fail"
  END IF
END
```

Step 6: Trace a real client–server flow

Example: Opening an **employee portal** in a browser.

1. Employee types URL → browser sends **GET request** to server
2. Server (Python/Django) checks session / shows login page → **HTML response**
3. Employee submits form → **POST request** with username and password
4. Server validates against **MySQL database** → sends dashboard or error
5. Browser renders the page — employee sees payslip link

```
Browser                    Server (Django)              Database (MySQL)
   |                            |                            |
```

```
    |--- POST /login ------------>|                            |
    |    (emp_id, password)       |--- SELECT employee ------->|
    |                             |<-- row or empty -----------|
    |<-- 200 dashboard HTML -----|                             |
```

Note:
- Wrong: Treating client and server as the same kind of “computer” with no distinction.
- Prefer: Client = *your* browser; server = the company’s remote machine.

Try these (2 programs):

Program 1 — Label the parts

```
# Campus canteen app (client–server)

– Client: Phone browser showing menu
– Request: GET /menu
– Server: Django app on cloud
– Database: MySQL table `menu_items`
– Response: HTML list of dosa, idli, chai with prices in ₹
```

Program 2 — Expense sync story

```
# Expense tracker sync

1. Ravi adds ₹120 chai on phone (client)
2. App sends POST /expenses JSON to server
3. Server INSERTs into MySQL
4. Friend Meera opens app → GET /expenses → sees Ravi's chai
Source of truth = server + database (not the phone alone)
```

Step 7: Write your career roadmap note

Create `notes/my_roadmap.md` in VS Code:

```
# My Python Full Stack Roadmap

– Goal role: Full Stack Developer (Python + Django + React)
– Timeline: 16 weeks
– Week 1–6: Core + Advanced Python + Git
– Week 7–11: MySQL + Frontend + Django start
– Week 12–16: Django labs + Flask + DRF + React
– Week 17–20: Testing, capstone, placement (AI optional)
– Portfolio target: Student management mini app
```

Try these (2 programs):

Program 1 — Role match note

```
# Role I want

Target: Full Stack Developer (Python)
City preference: Hyderabad / Bengaluru / remote
Why: Build both UI and APIs for campus/startup products
First portfolio app: Student marks + attendance mini site
```

Program 2 — Weekly habit plan

```
# Weekly habit (India IST)

- Mon–Fri: 1 live session + 45 min practice
- Sat: finish mini-project
- Sun: 5 GitHub commits + update roadmap
- Naukri keywords to watch: Django, MySQL, React, REST
```

Practice Exercises

Basic

1. Draw and label CPU, RAM, and storage on a blank laptop diagram.

Model answer: Sketch a laptop outline. Label **CPU** (processor — executes instructions), **RAM** (fast temporary working memory — apps currently open), and **storage** (SSD/HDD — permanent files, OS, projects). Mentally: CPU thinks, RAM holds today's desk papers, storage is the filing cabinet.

2. Create this folder tree using terminal only:

`ksr_python_fullstack/ ├── notes/ ├── projects/ └── labs/`

Model answer: From your Desktop (or training folder), run:

`bash mkdir -p ksr_python_fullstack/notes ksr_python_fullstack/projects ksr_python_fullstack/labs ls -R ksr_python_fullstack`

`mkdir -p` creates parent and child folders in one go. Confirm with `ls -R` that all three subfolders exist.

3. Write a 6-step algorithm for **withdrawing cash from an ATM** (no code).

Model answer:
1. Insert card / select account.
2. Enter PIN and authenticate.
3. Choose "Withdraw" and enter amount.
4. Check balance ≥ amount; if not, show insufficient funds and stop.
5. Dispense cash and update balance.
6. Print/show receipt and return card (or end session).

4. List five file extensions you expect to use in this course and what each is for.

Model answer:
- `.py` — Python source files
- `.md` — Markdown notes / README
- `.html` / `.css` / `.js` — frontend pages, styles, browser scripts
- `.sql` — MySQL scripts
- `.json` — API / config data (also `.gitignore`, `.txt` for plain notes)

Intermediate

1. On paper, draw client–server flow for an **expense tracker mobile app** syncing with a server.

Model answer: Phone app = **client**. Cloud/API + database = **server**. Flow: User adds expense → client sends HTTP request (JSON: amount, category, date) → server validates and stores in DB → server responds success/error → client updates local UI. Sync later: client pulls latest expenses from server on login/open.

2. Open VS Code, install Python extension, create `notes/glossary.md` with 10 terms from this module.

Model answer: Install Microsoft **Python** extension. Create `notes/glossary.md` with terms such as: CPU, RAM, storage, OS, algorithm, program, client, server, frontend, backend, terminal, IDE, full stack, HTTP, database. One-line plain-English definition each.

3. Compare two job posts on Naukri/LinkedIn for "Python Developer" (entry-level) — note required skills vs this course map.

Model answer: Typical entry posts ask: Python basics, Git, SQL/MySQL, Django/Flask, REST APIs, HTML/CSS/JS, sometimes React. Map to this course: Foundations → Core Python → Git → MySQL → Frontend → Django/Flask → APIs → React → Testing/Placement. Soft skills (communication, problem-solving) appear often — practice explaining projects in plain English.

4. Write pseudocode for **task manager**: add task, list tasks, mark complete, exit.

Model answer:

```
tasks = empty list
LOOP forever
  show menu: 1 Add  2 List  3 Complete  4 Exit
  read choice
  IF choice == 1: read title; append to tasks as incomplete
  IF choice == 2: print each task with status
  IF choice == 3: read task number; mark that task complete
  IF choice == 4: stop
```

Interview-Oriented

1. Explain client–server model to a non-technical friend in 60 seconds.

Model answer: "Your phone is the **client** — it asks for data. The company's computer on the internet is the **server** — it stores and replies. Like ordering food: you (client) place the order; the kitchen (server) prepares and sends it back. WhatsApp chat, Flipkart pages, and banking apps all work this way."

2. What is the difference between an algorithm and a program? Give one example each.

Model answer: An **algorithm** is a human-readable sequence of steps to solve a problem (e.g. steps to calculate monthly salary after PF deduction). A **program** is that algorithm written in a language a computer can run (e.g. `salary.py` in Python that performs those steps).

3. Name three differences between frontend and backend with a real website example.

Model answer: Using Flipkart/Amazon-style shopping: (1) **Frontend** — product page layout, buttons, CSS you see in the browser; **backend** — stock check, price calculation, order save on the server. (2) Frontend runs mainly in the browser; backend runs on servers. (3) Frontend languages often HTML/CSS/JS/React; backend often Python/Django + MySQL. Users see frontend; they don't see the database directly.

4. Why do developers use terminal when GUI (file explorer) exists?

Model answer: Many servers have no GUI. Git, Python, SSH, and deployment tools are CLI-first. Terminal commands are scriptable and repeatable (same steps every time). Interviewers expect comfort with `cd`, `ls` / `dir`, `python3`, and `git` — not only clicking folders.

Mini-Project (Assignment): Personal Developer Workspace Setup

Lab kit (build here): 15_Lab_Assets/27_terminal_workspace/

Goal: Leave the session with a repeatable workspace.

What to build

1. Create `ksr_python_fullstack` with subfolders: `notes`, `projects`, `labs`
2. Add `notes/day01_foundations.md` summarizing today's concepts in your own words
3. Add `notes/my_roadmap.md` with target role and 16-week milestones
4. Screenshot: VS Code with folder tree visible + terminal showing `pwd` and `ls`
5. Submit screenshot + both markdown files (Google Drive / LMS)

Stretch

- Add `labs/day01_workspace/CHECKLIST.md` copied from the lab kit starter.
- Pin your course root folder in VS Code for one-click reopen.
- Write one Naukri/LinkedIn job note: 3 skills from this course that match an entry-level Python post.

Done when

- [ ] Folder tree `notes/`, `projects/`, `labs/` exists and opens in VS Code
- [ ] `notes/day01_foundations.md` and `notes/my_roadmap.md` are filled in your words
- [ ] Terminal shows correct `pwd` / `ls` for the course root
- [ ] Lab kit ticket steps completed

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. If the internet stops working, can a client still show pages already loaded? Why or why not?
2. Is WhatsApp on your phone the client, the server, or both? Explain.
3. Why might a company in Bangalore hire separate frontend and backend teams?
4. What could go wrong if two developers save files in different folder structures?
5. Looking at retail or banking backgrounds, which developer role feels closest to your past work — and why?

Q1. Explain what happens when you type a URL and press Enter.
A: Browser (client) sends HTTP request to server at that URL; server processes and returns HTML/resources; browser renders the page. DNS first resolves domain name to server IP.

Q2. What is the difference between hardware and software?
A: Hardware is physical (CPU, RAM); software is programs and data (OS, Python, VS Code). Software runs on hardware.

Q3. What does a full stack developer do?
A: Builds both user-facing part (frontend) and server/database part (backend) of an application, and often deploys it — end-to-end product development.

Q4. Why learn terminal if we have VS Code GUI?
A: Servers often have no GUI; Git, Python, and deployment tools are CLI-first; terminal is faster for automation and industry standard.

Q5. Algorithm vs program — example?
A: Algorithm: steps to calculate employee monthly tax. Program: those steps written in Python and saved as `tax_calculator.py` that machine executes.
